Демо 3. Продвинутый CLAUDE.md: рефакторинг и Progressive Disclosure

Что делаем

Рефакторим CLAUDE.md из Ступени 1. На первом занятии мы написали монолитный файл с правилами проекта, стека, стиля статей и рабочего процесса. Сейчас он разросся и содержит правила, которые нужны Claude не всегда, а только при работе с определёнными файлами. Это Context Bloat: каждый токен в CLAUDE.md загружается при старте и съедает контекстное окно.

Цель: превратить 150-строчный CLAUDE.md в компактный 40-строчный файл с модульными правилами в `.claude/rules/`, добавить Progressive Disclosure и Compact Instructions.

Подготовка

1. Репозиторий дайджеста с CLAUDE.md из Ступени 1 (содержит правила стека, команд, стиля статей, Git-процесса).
2. Claude Code установлен и работает.
3. Пройдено Демо 1 (управление контекстом) — вы знаете, что показывает `/context`.

Шаг 1. Аудит текущего CLAUDE.md

Перед рефакторингом нужно понять, что в CLAUDE.md лишнее. Попросите Claude проанализировать файл:

```
Проанализируй наш CLAUDE.md. Для каждой секции ответь:
1. Это нужно Claude всегда или только при работе с определёнными файлами?
2. Это правило, которое Claude нарушит без напоминания, или очевидная
```

```
конвенция?
3. Может ли Claude узнать это из кода (package.json, tsconfig, структура
папок)?
```

Claude выдаст что-то вроде:

```
Секция "Stack" — Claude может узнать стек из package.json. Оставить только
неочевидное.
Секция "Article Style" — нужна только при работе с content/*.md. Вынести.
Секция "Git Workflow" — нужна только при коммитах. Вынести.
Правило "ES modules" — Claude увидит import/export в существующем коде.
Убрать.
```

На что обратить внимание: правила делятся на три категории.

Категория	Пример	Решение
Всегда нужно	Команды сборки, архитектура проекта	Оставить в CLAUDE.md
Нужно для конкретных файлов	Стиль статей, правила тестов	Вынести в `.claude/rules/` с `paths:`
Очевидно из кода	«Используем TypeScript», «ES modules»	Удалить

Шаг 2. Создаём .claude/rules/ с фильтром paths:

Файлы в `.claude/rules/` загружаются автоматически при старте сессии. Но если добавить frontmatter с `paths:`, файл загрузится только при работе с файлами по указанному glob-паттерну. Это экономит контекст.

Создайте папку и первый файл правил:

```
mkdir -p .claude/rules
```

Попросите Claude вынести правила статей:

```
Вынеси из CLAUDE.md все правила, касающиеся написания статей, в файл
.claude/rules/article-style.md. Добавь frontmatter с paths: для файлов
контента. В CLAUDE.md оставь только ссылку.
```

Claude создаст файл `.claude/rules/article-style.md`:

```
---
paths:
  - "src/content/**/*.md"
  - "src/content/**/*.mdx"
---

# Стиль статей дайджеста

- Язык: русский.
- Заголовок: информативный, без кликбейта, до 80 символов.
- Первый абзац — суть новости в 2-3 предложениях.
- Обязательна ссылка на первоисточник.
- Стиль: информационный. Без маркетинговых клише, без восклицательных
знаков.
- Объём: 800-1200 слов.
- Структура: заголовок → суть → детали → почему это важно → ссылки.
```

Как работает paths:

Frontmatter `paths:` принимает glob-паттерны. Правила из файла загрузятся в контекст, только если Claude работает с файлами, которые совпадают с паттерном.

Примеры glob-паттернов для `.claude/rules/`:

Паттерн	Что совпадает
`src/content/**/*.md`	Все markdown-файлы в любой вложенности внутри `src/content/`
`tests/**`	Все файлы в папке `tests/`
`*.config.*`	Файлы конфигурации в корне: `astro.config.mjs`, `tailwind.config.ts`
`src/components/**/*.astro`	Все Astro-компоненты

`**/*.test.ts`	Все тест-файлы в проекте

Создайте ещё один файл — `.claude/rules/git-workflow.md`:

```
---
paths:
  - "**"
---

# Git-процесс

- Ветки: feature/название, fix/название.
- Коммиты: conventional commits на русском — feat(область): описание.
- Перед коммитом: npm run build && npm run typecheck.
- Никогда не коммить в main напрямую.
```

Здесь `paths: ["**"]` означает «при работе с любыми файлами» — эквивалент отсутствия paths:, но явный.

Шаг 3. Compact Instructions — что сохранять при сжатии

Когда контекст заполняется, Claude автоматически сжимает историю разговора. Или вы вызываете `/compact` вручную. Проблема: при сжатии могут потеряться важные решения, принятые в ходе сессии.

Compact Instructions — секция в CLAUDE.md, которая говорит Claude, что обязательно сохранить при компрессии.

Добавьте в CLAUDE.md:

```
Попроси Claude добавить в CLAUDE.md секцию Compact Instructions. В ней
должны быть пункты: какие решения сохранять, какой контекст критичен, в
каком формате писать сжатое резюме.
```

Результат — секция в CLAUDE.md:

```
# Compact Instructions
```

```
При сжатии контекста обязательно сохрани:
- Список уже написанных статей (заголовок + файл).
- Текущую задачу и её статус.
- Принятые архитектурные решения текущей сессии.
- Ошибки, которые уже были найдены и исправлены.

Формат сжатого резюме:
- Написано: [список файлов].
- В процессе: [текущая задача].
- Решения: [ключевые решения].
```

Проверка. Загрузите контекст чем-нибудь (прочитайте несколько файлов, попросите Claude проанализировать код), затем выполните:

```
/compact "работаем над рефакторингом CLAUDE.md"
```

Проверьте `/context` — объём должен уменьшиться. Проверьте, что Claude помнит контекст сессии в сжатом виде.

Шаг 4. Progressive Disclosure: @import vs «See file for details»

Два способа ссылаться на внешние файлы из CLAUDE.md. Они работают принципиально по-разному.

Сравнение

	`@import`	«See file for details»
Синтаксис	`@path/to/file.md`	«Правила стиля статей описаны в `.claude/rules/article-style.md`.»
Когда загружается	При старте сессии. Содержимое файла вставляется в контекст сразу.	Когда Claude решит, что эти правила нужны. Файл читается по требованию.

Расход контекста	Постоянный. Файл всегда в контексте.	Только при необходимости.
Гарантия применения	Высокая. Правила всегда перед глазами.	Средняя. Claude может забыть прочитать файл.
Когда использовать	Критичные правила, которые нельзя пропустить.	Справочная информация, редко используемые правила.

Практика

Замените в CLAUDE.md инлайн-правила на ссылки. Используйте формулировку, которая побуждает Claude читать файл:

```
Перепиши CLAUDE.md. Замени инлайн-секции на ссылки:
- Для критичных правил используй @import.
- Для справочных — фразу "Before working on [topic], read [file] for current rules."
```

Пример результата в CLAUDE.md:

```
# Article Style

Before writing or editing articles in src/content/, read .claude/rules/article-style.md for current editorial rules.
```

Важно: формулировка «Before working on... read...» работает лучше, чем просто «See file». Она даёт Claude чёткий триггер — когда именно нужно прочитать файл.

Шаг 5. Результат: до и после

CLAUDE.md до рефакторинга (~150 строк)

```
# AI Digest Blog

Блог с автоматическими дайджестами новостей AI.
```

Stack

- Astro 5 — статический генератор.
- TypeScript — строгий режим.
- Tailwind CSS 4.
- Vercel — деплой.
- ES modules (import/export).

Commands

- npm run dev — dev-сервер на :4321.
- npm run build — production-сборка.
- npm run preview — предпросмотр.
- npm run typecheck — проверка типов.

Article Style

- Язык: русский.
- Заголовок: информативный, без кликбейта, до 80 символов.
- Первый абзац — суть новости в 2-3 предложениях.
- Обязательна ссылка на первоисточник.
- Стиль: информационный. Без маркетинговых клише.
- Без восклицательных знаков в тексте.
- Объём: 800-1200 слов.
- Структура: заголовок → суть → детали → почему это важно → ссылки.
- Мета-описание: до 160 символов.
- Теги: 3-5 тегов из списка в src/data/tags.ts.
- Дата публикации: формат YYYY-MM-DD.
- Slug: транслитерация заголовка, через дефис.

Git Workflow

- Ветки: feature/название, fix/название.
- Коммиты: conventional commits на русском.
- Формат: feat(область): описание.
- Перед коммитом: npm run build && npm run typecheck.
- Никогда не коммить в main напрямую.
- PR через GitHub CLI: gh pr create.

Testing

- Тесты: vitest.
- Файлы тестов: *.test.ts в папке tests/.
- *Моки внешних сервисов обязательны.*
- *Запуск: npm run test.*

Content Structure

```
- Статьи: src/content/posts/*.md.
- Изображения: public/images/posts/.
- Компоненты: src/components/.
- Layouts: src/layouts/.
- Pages: src/pages/.
```

CLAUDE.md после рефакторинга (~40 строк)

```
# AI Digest Blog

Блог с автоматическими дайджестами новостей AI. Astro 5, TypeScript,
Tailwind CSS 4, Vercel.

# Commands

- npm run dev — dev-сервер на :4321.
- npm run build — production-сборка.
- npm run typecheck — проверка типов.

# Architecture

- Статьи: src/content/posts/*.md.
- Компоненты: src/components/.
- Layouts: src/layouts/.

# Article Style

Before writing or editing articles in src/content/, read
.claude/rules/article-style.md for current editorial rules.

# Git & Testing

Before committing code, read .claude/rules/git-workflow.md for branch and
commit conventions.
Before writing tests, read .claude/rules/testing.md for test structure
and conventions.

# Compact Instructions

При сжатии контекста обязательно сохрани:
- Список уже написанных статей (заголовок + файл).
- Текущую задачу и её статус.
- Принятые архитектурные решения текущей сессии.
- Ошибки, которые уже были найдены и исправлены.
```

Итого: 150 строк стали 40. Правила никуда не пропали — они в `.claude/rules/` и загружаются по требованию.

Структура файлов после рефакторинга:

```
project/
├── CLAUDE.md                          # 40 строк — ядро
├── .claude/
│   └── rules/
│       ├── article-style.md           # paths: src/content/**/*.md
│       ├── git-workflow.md            # paths: **
│       └── testing.md                 # paths: tests/**
```

Шаг 6. Проверка результата

6.1. Проверка загрузки контекста

Запустите Claude Code и выполните:

```
/context
```

Посмотрите на объём загруженного контекста. Сравните с тем, что было до рефакторинга. CLAUDE.md теперь занимает меньше токенов при старте.

6.2. Проверка, что правила работают

Попросите Claude написать статью:

```
Напиши статью для дайджеста о том, что Anthropic выпустила Claude 4 Opus.
```

Claude должен прочитать `.claude/rules/article-style.md` перед написанием — вы увидите это в логе. Статья должна соответствовать всем правилам: русский язык, информационный стиль, структура, ссылки.

6.3. Проверка, что paths: фильтрует

Попросите Claude сделать что-то, не связанное со статьями:

```
Добавь в компонент Header.astro ссылку на страницу "О проекте".
```

Claude не должен загружать `article-style.md` — эта задача не затрагивает `src/content/**/*.md`.

6.4. Проверка Compact Instructions

Загрузите контекст (прочитайте несколько файлов, задайте вопросы), затем:

```
/compact "рефакторинг CLAUDE.md"
```

После сжатия спросите:

```
Какие файлы мы сегодня меняли?
```

Claude должен ответить из сжатого резюме — Compact Instructions сработали.

Чеклист

- [ ] Проведён аудит CLAUDE.md — определены лишние, очевидные и контекстно-зависимые правила.
- [ ] Создана папка `.claude/rules/` с файлом `article-style.md` и frontmatter `paths:`.
- [ ] В CLAUDE.md добавлена секция Compact Instructions.
- [ ] Инлайн-правила заменены на ссылки с Progressive Disclosure (формулировка «Before working on... read...»).
- [ ] `/context` показывает уменьшение занятого контекста.
- [ ] Claude корректно применяет правила из `.claude/rules/` при работе с соответствующими файлами.
- [ ] `/compact` сохраняет критичный контекст согласно Compact Instructions.